



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Karina García Morales

Asignatura: Fundamentos de Programación

Grupo: 22

No de Práctica(s): Practica 6

Integrante(s): Roja Martínez Francisco Alexander

*No. de lista o
brigada:* 40

Semestre: 2026-1

Fecha de entrega: 15-10-2025

Observaciones:

CALIFICACIÓN:

Entorno y fundamentos del lenguaje C

Objetivo:

El alumnado elaborará programas en lenguaje C utilizando las instrucciones de control de tipo secuencia, para realizar la declaración de variables de diferentes tipos de datos, así como efectuar llamadas a funciones externas de entrada y salida para asignar y mostrar valores de variables y expresiones.

Desarrollo:

El software tiene un ciclo en el hasta el momento sólo habíamos llegado a la parte del pseudocódigo, ahora pasaremos a la etapa de codificación y las pruebas.



Figura 1: Ciclo de vida del software.

Entorno de C

El lenguaje C es uno de los lenguajes de programación más importantes.

¿Por qué es tan usado?

C es demasiado poderoso porque combina dos cosas:

1. Es fácil de usar (como un lenguaje de alto nivel).
2. Permite controlar la máquina a detalle (como un lenguaje de bajo nivel).

Por eso, se usa para programar miles de aparatos electrónicos en todo el mundo.

¿Cómo funciona?

1. Escribes las instrucciones del programa en un archivo de texto (el código fuente).

2.Un programa llamado compilador toma ese texto y lo convierte en un programa ejecutable que la computadora puede entender y correr.

Editores

Para crear un programa en C, primero hay que escribir el código en un editor de texto. El editor y el compilador que elijas dependerán de tu sistema operativo (Windows, Mac, Linux) y de si prefieres usar una interfaz gráfica o no. Es crucial entender que un editor de texto solo maneja texto plano (sin formato), ideal para código que será leído por el compilador. En cambio, un procesador de texto (como Word) añade formato, imágenes y otras etiquetas que solo estorbarían y confundirían al compilador. Por eso, para programar en C, solo necesitamos un editor de texto simple.

Editor Visual Interface de GNU/Linux (vi)

Con este comando (Vi), podremos crear y nombrar un archivo

Para iniciar *vi*, debe teclearse desde la línea de comandos:

```
vi nombre_archivo[.ext]
```

En la práctica lo aplicamos para nombrar nuestros programas

```
compilation terminated.  
Granada64:~ fp22alu40$ vi programa1.c  
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out  
Granada64:~ fp22alu40$
```

Modo comando

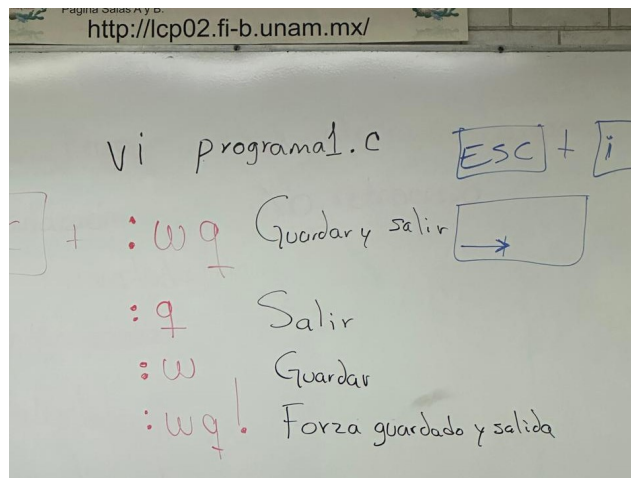
- ↑ o k mueve el cursor hacia arriba.
- ↓ o j mueve el cursor hacia abajo.
- ← o h mueve el cursor hacia la izquierda.
- → o l mueve el cursor hacia la derecha.
- 1G lleva el cursor al comienzo de la primera línea.
- G lleva el cursor al comienzo de la última línea.
- x borra el carácter marcado por el cursor.
- dd borra o corta la línea donde está el cursor.

- nnd donde n es la cantidad de líneas que se borrarán o cortarán después del cursor.
- D borra o corta desde la posición de cursor hasta el final de la línea.
- dw borra o corta desde la posición del cursor hasta el final de una palabra.
- yy copia la línea donde está el cursor.
- p pega un contenido copiado o borrado.
- u deshace el último cambio.

Modo de última línea

- /texto donde la cadena texto será buscada hacia delante de donde se encuentra el cursor.
- ?texto donde la cadena texto será buscada hacia atrás de donde se encuentra el cursor.
- :q para salir de vi sin haber editado el texto desde la última vez que se guardó.
- :q! para salir de vi sin guardar los cambios.
- :w para guardar los cambios sin salir de vi.
- :w archivo para realizar la orden "guardar como", siendo archivo el nombre donde se guardará el documento.
- :wq guarda los cambios y sale de vi.
- :wq! Forzar guardado y salida

Los elementos de la foto son comandos que usamos en la práctica, Los cuales también especifican su función.



Compiladores

Una vez codificado un programa en C en algún editor de texto, éste debe ser leído por un programa que produzca un archivo ejecutable. A este programa se le conoce como compilador.

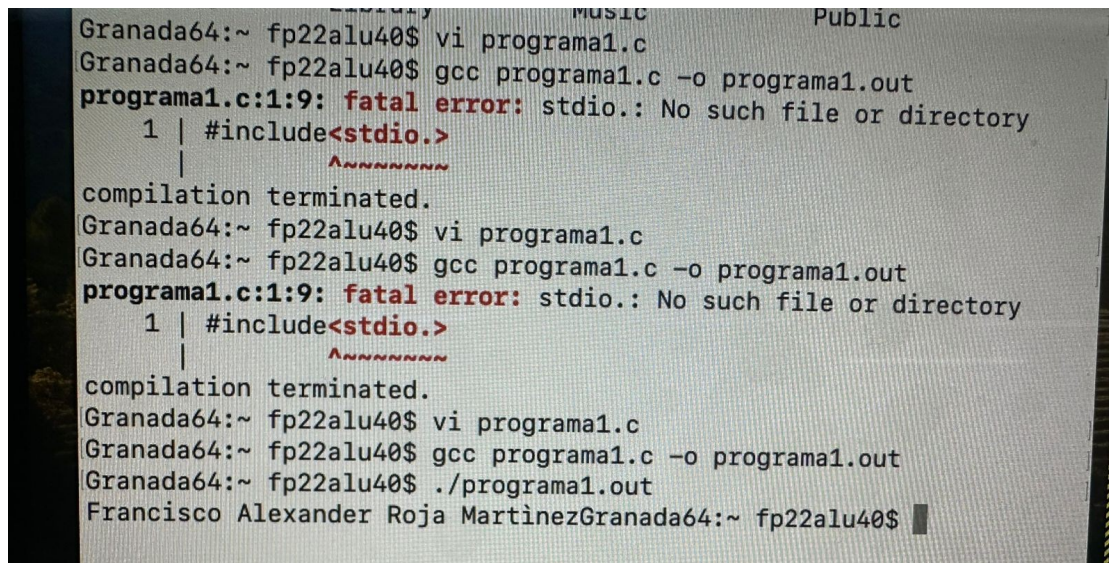
Compiladores gcc

En la práctica para compilar nuestros programas usamos los compiladores gcc.

Con la sintaxis siguiente

gcc calculadora.c -o calculadora.out

En forma de que se escribe gcc nombre del archivo con un guion (-) un punto (.) y un out.



```
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
programa1.c:1:9: fatal error: stdio.: No such file or directory
  1 | #include<stdio.>
    |             ^
compilation terminated.
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
programa1.c:1:9: fatal error: stdio.: No such file or directory
  1 | #include<stdio.>
    |             ^
compilation terminated.
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
Granada64:~ fp22alu40$ ./programa1.out
Francisco Alexander Roja MartinezGranada64:~ fp22alu40$
```

Ejecución

La ejecución es la etapa que sigue después de haber compilado el programa. Y este nos ayuda a dar una vista previa de lo que hemos codificado.

Hay una forma para ejecutar el programa de la siguiente forma y con este comando.

`./calculadora.out`

La cual va con la siguiente nomenclatura punto (.) barra inclinada o slash (/), seguido por el nombre del archivo y por ultimo un .out

Un ejemplo dentro de la actividad en clase de la practica son los siguientes.


```
compilation terminated.  
Granada64:~ fp22alu40$ vi programa1.c  
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out  
Granada64:~ fp22alu40$ ./programa1.out  
Francisco Alexander Roja MartínezGranada64:~ fp22alu40$
```

Comentarios

Los comentarios son únicamente para el programador, puesto que el solo los puede ver, dentro de la codificación, siempre y cuando se use la nomenclatura correcta, para agregar estos comentarios

El comentario por línea inicia cuando se insertan los símbolos // y termina con el salto de línea (hasta donde termine el renglón).

Dentro de la practica creamos un archivo donde se hace uso de los comentarios y lo nombramos vi Programa1.c

```
fp22alu40 — vi programa1.c — 68x35  
//Este es mi primer programa  
#include<stdio.h>  
int main (){  
    printf("Francisco Alexander Roja Martínez");  
    /*Este es  
    un  
    comentario  
    de parrafo*/  
    return 0;  
}
```

El conjunto de caracteres en C

Del mismo modo que en nuestro lenguaje habitual utilizamos un conjunto de caracteres para construir instrucciones que tengan significado, los programas que se realicen en C se escriben utilizando un conjunto de caracteres formado por lo siguiente:

- Las 26 letras minúsculas del alfabeto inglés (a b c d e f g h i j k l m n o p q r s t u v w x y z).
- Las 26 letras mayúsculas (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z).
- Los 10 dígitos (1 2 3 4 5 6 7 8 9 0).

• Los símbolos especiales (@ ^ { } [] () & \$ % # ~ ' " / ? ; : _ . , = / * - +). • El espacio en blanco o barra espaciadora. Nota: Lenguaje C es sensible a minúsculas y mayúsculas.

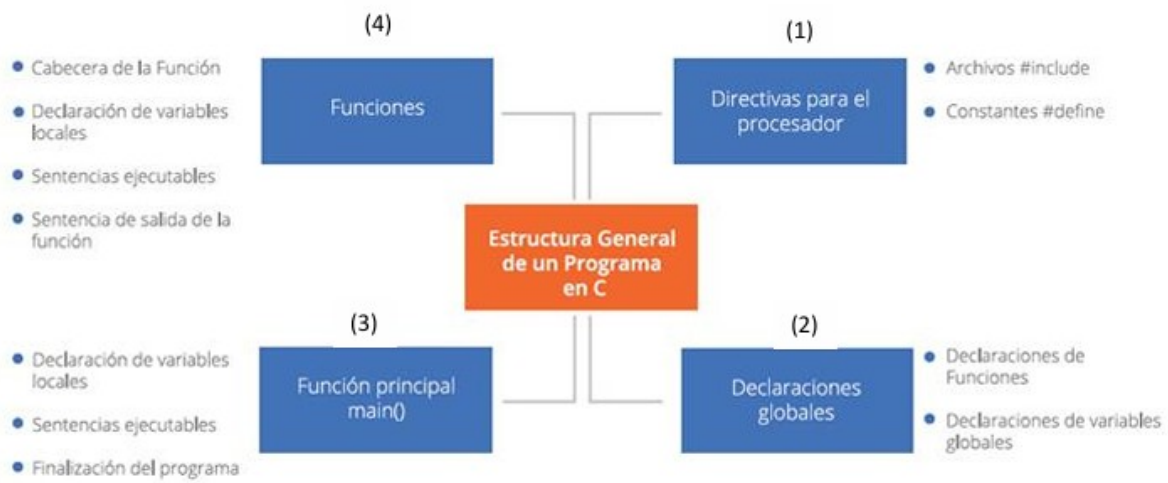


Figura 4: Estructura General de un programa en C

Palabras reservadas

Tabla 1: Palabras reservadas

<i>auto</i>	<i>double</i>	<i>int</i>	<i>struct</i>
<i>break</i>	<i>else</i>	<i>long</i>	<i>switch</i>
<i>case</i>	<i>enum</i>	<i>register</i>	<i>typedef</i>
<i>char</i>	<i>extern</i>	<i>return</i>	<i>union</i>
<i>const</i>	<i>float</i>	<i>short</i>	<i>unsigned</i>
<i>continue</i>	<i>for</i>	<i>signed</i>	<i>void</i>
<i>default</i>	<i>goto</i>	<i>sizeof</i>	<i>volatile</i>
<i>do</i>	<i>if</i>	<i>static</i>	<i>while</i>

Tipos de datos

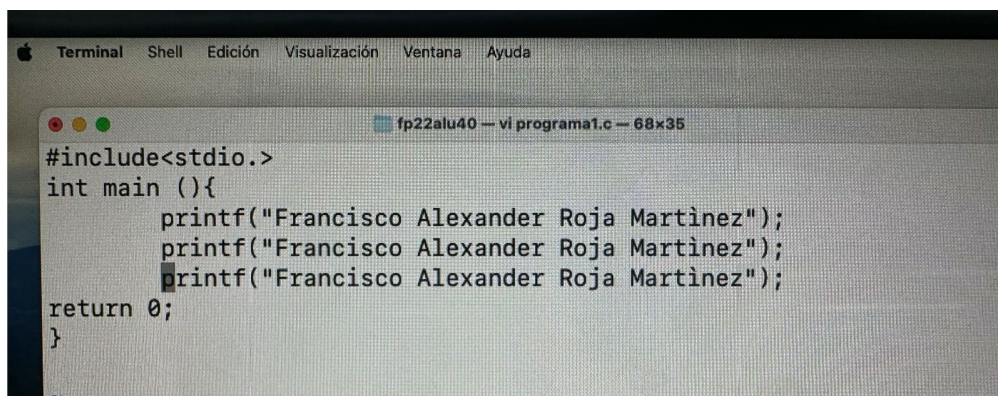
Tabla 2: Tipos de datos.

Tipo	Descripción	Espacio en Memoria	Rango
<i>int</i>	Cantidad entera	2 bytes o una palabra* (varía de un compilador a otro)	-32767 a 32766
<i>char</i>	Carácter	1 byte	-128 a 127
<i>float</i>	Número en punto flotante (un número que incluye punto decimal y/o exponente)	1 palabra* (4 bytes)	$-3.4E^{38}$ a $3.4 E^{38}$
<i>double</i>	Número en punto flotante de doble precisión (más cifras significativas y mayor valor posible del exponente)	2 palabras* (8 bytes)	$-1.7E^{308}$ a $1.7E^{308}$

Código que ejemplifica el uso de la función printf.

En este programa usamos algunos comandos del segundo programa, pero en esta ocasión el programa 3 usamos el comando printf el cual es el escribir de nuestro pseudocódigo y imprécamos del diagrama de flujo, como bien lo mencionamos antes este solo va a imprimir en pantalla lo que le escribimos

De igual manera con ayuda de la práctica sólo tuvimos que compilar y ejecutar.



```

Terminal  Shell  Edición  Visualización  Ventana  Ayuda
fp22alu40 — vi programa1.c — 68x35
#include<stdio.>
int main (){
    printf("Francisco Alexander Roja Martínez");
    printf("Francisco Alexander Roja Martínez");
    printf("Francisco Alexander Roja Martínez");
    return 0;
}

```



```

compilation terminated.
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
Granada64:~ fp22alu40$ ./programa1.out
Francisco Alexander Roja MartínezGranada64:~ fp22alu40$

```

Tabla 4: Secuencias de caracteres

Especificador	Descripción
<code>\a</code>	Carácter de alarma
<code>\b</code>	Retroceso
<code>\f</code>	Avance de hoja
<code>\n</code>	Salto de línea
<code>\r</code>	Regreso de carro
<code>\t</code>	Tabulador horizontal
<code>\v</code>	Tabulador vertical
<code>\0</code>	Carácter nulo

Tabla 5: Caracteres de conversión de los datos de entrada de uso común [tabla], basada en Gottfried, 1997, *Programación en C* (p. 92).

Carácter de conversión	Significado
<code>c</code>	El dato es un carácter
<code>d</code>	El dato es un entero decimal
<code>e</code>	El dato es un valor en coma flotante
<code>f</code>	El dato es un valor en coma flotante
<code>g</code>	El dato es un valor en coma flotante

<code>h</code>	El dato es un entero corto
<code>i</code>	El dato es un entero decimal, octal o hexadecimal
<code>o</code>	El dato es un entero octal
<code>s</code>	El dato es una cadena de caracteres seguida de un carácter de espaciado (se añade automáticamente el carácter nulo <code>\0</code> al final)
<code>u</code>	El dato es un entero sin signo
<code>x</code>	El dato es un entero hexadecimal

Código que ejemplifica el uso de la función scanf

El siguiente programa muestra el almacenamiento de datos en variables. Compila y ejecuta bien.

En esta ocasión usamos lo que aplicamos en los tres programas anteriores, a excepción de uno que es scanf el cual nos ayuda a compilar los datos que el usuario nos da para continuar con el proceso de nuestro programa.

Operadores aritméticos

Tabla 6: Operadores aritméticos

Operador	Operación	Uso	Resultado
+	Suma	125.78 + 62.5	188.28
-	Resta	65.3 - 32.33	32.97
*	Multiplicación	8.27 * 7	57.75
/	División	15 / 4	3.75
%	Módulo	4 % 2	0

Expresiones lógicas

Tabla 7: Operadores relacionales

Operador	Operación	Uso	Resultado
==	Igual que Equivalente a	'h' == 'H'	Falso
!=	Diferente a	'a' != 'b'	Verdadero
<	Menor que	7 < 15	Verdadero
>	Mayor que	11 > 22	Falso
<=	Menor o igual	15 <= 22	Verdadero
>=	Mayor o igual	20 >= 35	Falso

Tabla 8: Operadores lógicos

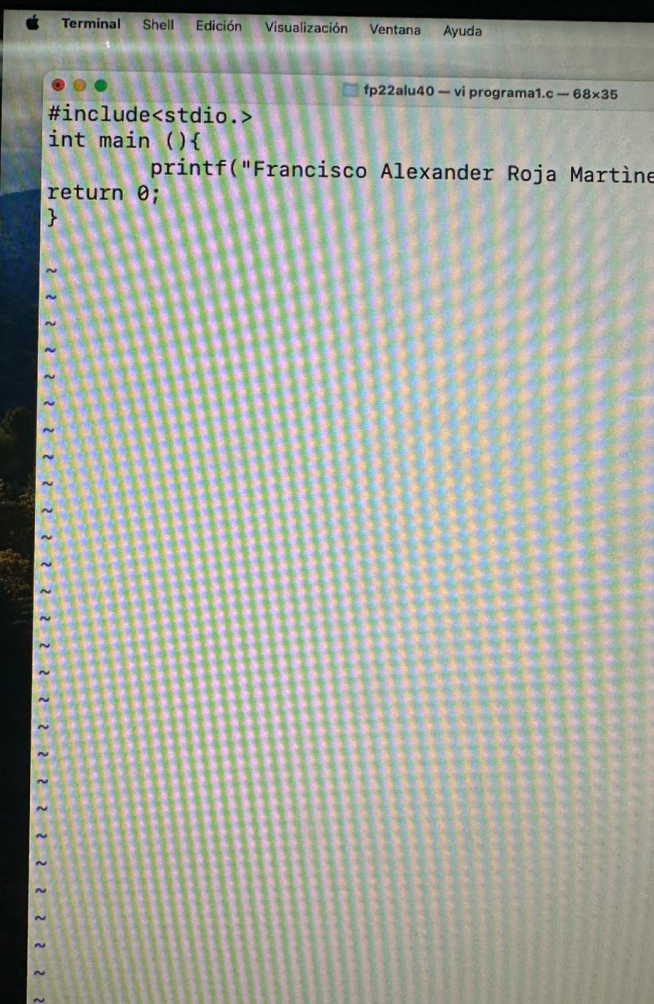
Operador	Operación	Uso
!	No	! p
&&	Y	a > 0 && a < 11
	O	opc == 1 salir != 0

Otros operadores (avanzado).

Tabla 9: Operadores a nivel de bits (avanzado)

Operador	Operación	Uso	Resultado
>>	Corrimiento a la derecha	8 >> 2	2
<<	Corrimiento a la izquierda	8 << 1	16
&	Operador AND	5 & 4	4
	Operador OR	3 2	3
~	Complemento ar-1	~2	1

Ejercicio en clase



The image shows a macOS Terminal window with the menu bar at the top containing 'Terminal', 'Shell', 'Edición', 'Visualización', 'Ventana', and 'Ayuda'. The title bar of the window reads 'fp22alu40 — vi programa1.c — 68x35'. Inside the terminal, a C program is being edited in vi. The code is as follows:

```
#include<stdio.>
int main (){
    printf("Francisco Alexander Roja Martínez");
return 0;
}
```

Below the code, there are several lines of tilde (~) characters, indicating the end of the file or buffer. At the bottom left, the prompt ':wq' is visible, indicating the user is in vi command mode and has pressed the 'w' key to write the file.


```
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
programa1.c:1:9: fatal error: stdio.: No such file or directory
  1 | #include<stdio.>
    |             ^~~~~~
compilation terminated.
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
programa1.c:1:9: fatal error: stdio.: No such file or directory
  1 | #include<stdio.>
    |             ^~~~~~
compilation terminated.
Granada64:~ fp22alu40$ vi programa1.c
Granada64:~ fp22alu40$ gcc programa1.c -o programa1.out
Granada64:~ fp22alu40$ ./programa1.out
Francisco Alexander Roja MartìnezGranada64:~ fp22alu40$
```

```
fp22alu40 — vi programa1.c — 68x35
//Este es mi primer programa
#include<stdio.h>
int main (){
    printf("Francisco Alexander Roja Martìnez");
    /*Este es
un
comentario
de parrafo*/
return 0;
}
```



```
Terminal Shell Edición Visualización Ventana Ayuda
fp22alu40 — vi programa3.c — 82x35

#include <stdio.h>
int main() {
    //Declaración de variables
    int entero;
    float flotante;
    double doble;
    char caracter;
    //Asignación de variables
    entero = 14;
    flotante = 3.5f;
    doble = 6.8e10;
    caracter = 'A';

    //Funciones de salida de datos en pantalla
    printf("La variable entera tiene valor: %i \n", entero);
    printf("La variable flotante tiene valor: %f \n", flotante);
    printf("La variable doble tiene valor: %f \n", doble);
    printf("La variable caracter tiene valor: %c \n", caracter);

    printf("Entero como octal: %o \n Como Hexadecimal %X \n", entero, entero);
    printf("Flotante con precisión: %5.2f \n", flotante);
    printf("Doble con precisión: %5.2f \n", doble);
    printf("Carácter como entero: %d \n", caracter);

    return 0;
}
```

```
Granada64:~ fp22alu40$ vi programa3.c
Granada64:~ fp22alu40$ vi programa3.c
Granada64:~ fp22alu40$ gcc programa3.c -o programa3.out
Granada64:~ fp22alu40$ vi programa3.c
Granada64:~ fp22alu40$ gcc programa3.c -o programa3.out
Granada64:~ fp22alu40$ ./programa3.out
La variable entera tiene valor: 14
La variable flotante tiene valor: 3.500000
La variable doble tiene valor: 68000000000.000000
La variable caracter tiene valor: A
Entero como octal: 16
Como Hexadecimal E
Flotante con precisión: 3.50
Doble con precisión: 68000000000.00
Carácter como entero: 65
Granada64:~ fp22alu40$
```



```

fp22alu40 - vi programa5.c - 85x30
#include <stdio.h>

int main()
{
    int enteroNumero;
    char caracterA = 65; // Convierte el entero a carácter ASCII.
    double puntoFlotanteNumero;

    // Asignar valor de teclado a una variable.
    printf("Escriba un valor entero: ");
    scanf("%i", &enteroNumero);
    printf("Escriba un valor real: ");
    scanf("%lf", &puntoFlotanteNumero);

    // Imprimir valores con formato.
    printf("\nImprimiendo las variables \a\n");
    printf("\t Valor de enteroNumero = %i \a\n", enteroNumero);
    printf("\t Valor de caracterA = %c \a\n", caracterA);
    printf("\t Valor de puntoFlotanteNumero = %lf \a\n",
        puntoFlotanteNumero);

    printf("\t Valor de enteroNumero en base 16 = %x \a\n", enteroNumero);
    printf("\t Valor de caracterA en código hexadecimal = %x\n", caracterA);
    printf("\t Valor de puntoFlotanteNumero\n");
    printf("en notación científica = %e\n", puntoFlotanteNumero);

    return 0;
}
"programa5.c" 31L, 881B

```

```

Granada64:~ fp22alu40$ vi programa5.c
Granada64:~ fp22alu40$ gcc programa5.c -o programa5.out
Granada64:~ fp22alu40$ ./programa5.out
Escriba un valor entero: 1
Escriba un valor real: 1.3

Imprimiendo las variables
    Valor de enteroNumero = 1
    Valor de caracterA = A
    Valor de puntoFlotanteNumero = 1.300000
    Valor de enteroNumero en base 16 = 1
    Valor de caracterA en código hexadecimal = 41
    Valor de puntoFlotanteNumero
en notación científica = 1.300000e+00
Granada64:~ fp22alu40$ █

```



```

fp22alu40 — vi programa6.c — 85x30
#include <stdio.h>
int main()
{
    short ocho, cinco, cuatro, tres, dos, uno;

    // 8 en binario: 0000 0000 0000 1000
    ocho = 8;
    // 5 en binario: 0000 0000 0000 0101
    cinco = 5;
    // 4 en binario: 0000 0000 0000 0100
    cuatro = 4;
    // 3 en binario: 0000 0000 0000 0011
    tres = 3;
    // 2 en binario: 0000 0000 0000 0010
    dos = 2;
    // 1 en binario: 0000 0000 0000 0001
    uno = 1;
    printf("Operadores aritméticos\n");
    printf("5 modulo 2 = %d\n", cinco%dos);
    printf("Operadores lógicos\n");
    printf("8 >> 2 = %d\n", ocho>>dos);
    printf("8 << 1 = %d\n", ocho<<1);
    printf("5 & 4 = %d\n", cinco&cuatro);
    printf("3 | 2 = %d\n", tres|dos);

    printf("\n");
    return 0;
}

```

```

Granada64:~ fp22alu40$ vi programa6.c
Granada64:~ fp22alu40$ gcc programa6.c -o programa6.out
Granada64:~ fp22alu40$ ./programa6.out
Operadores aritméticos
5 modulo 2 = 1
Operadores lógicos
8 >> 2 = 2
8 << 1 = 16
5 & 4 = 4
3 | 2 = 3

Granada64:~ fp22alu40$

```

Tarea

1.- Investigar cual es el dato que se encuentra por default en en lenguaje C(signed o unsigned) y mencionar las características con las que debe crearse una variable.

En el lenguaje C, el tipo de dato por defecto es "signed". Esto significa que, si no se especifica lo contrario, las variables de tipo entero pueden almacenar tanto números positivos como negativos.

Características:

- signed: Puede representar números positivos y negativos.
Ejemplo: -128 a 127 para signed char.
- unsigned: Solo representa números positivos, pero con el doble de rango positivo.
Ejemplo: 0 a 255 para unsigned char.

2.- Crea un programa en el que declares 4 variables haciendo uso de las reglas signed/unsigned, las cuatro variables deben ser solicitadas al usuario (se emplea scanf) y deben mostrarse en pantalla (emplear printf)

```
#include <stdio.h>
```

```
int main() {
```

```
    signed int num1;
```

```
    unsigned int num2;
```

```
    signed char letra;
```

```
    unsigned char codigo;
```

```
    printf("Ingresa un número con signo: ");
```

```
    scanf("%d", &num1);
```

```
    printf("Ingresa un número sin signo: ");
```

```
    scanf("%u", &num2);
```



```
printf("Ingresa un carácter con signo: ");
```

```
scanf(" %c", &letra);
```

```
printf("Ingresa un carácter sin signo: ");
```

```
scanf(" %c", &codigo);
```

```
printf("\n--- RESULTADOS ---\n");
```

```
printf("Número con signo: %d\n", num1);
```

```
printf("Número sin signo: %u\n", num2);
```

```
printf("Carácter con signo: %c\n", letra);
```

```
printf("Carácter sin signo: %c\n", codigo);
```

```
return 0;
```

```
}
```

3.- Comparación entre Editor de Texto y Procesador de Texto (Realizar una tabla comparativa)

<i>Característica</i>	<i>Característica</i>	<i>Editor de texto</i>	<i>Procesador de texto</i>
<i>Ejemplo</i>	Ejemplo	Notepad, VS Code	Word, Google Docs
<i>Función principal</i>	Función principal	Editar código o texto sin formato	Crear y dar formato a documentos
<i>Formato</i>	Formato	Texto plano (.txt, .c)	Texto enriquecido (.docx, .odt)
<i>Recursos</i>	Recursos	Usa pocos recursos del sistema	Requiere más recursos
<i>Enfoque</i>	Enfoque	Programación y desarrollo	Redacción y presentación visual

4.- Indica los comandos utilizados para compilar y para ejecutar un programa en iOS o Linux

Comandos en Linux (Terminal):

`gcc programa.c -o programa # Compilar`

`./programa # Ejecutar`

Comandos en macOS (iOS):

`clang programa.c -o programa # Compilar`

`./programa # Ejecutar`

5.- Indicar que sucede cuando en una variable tipo carácter se emplea el formato %d, %i, %o, %x

En C, un carácter (char) internamente se almacena como un número entero (código ASCII).

Por eso, al imprimir un carácter con diferentes formatos, se muestran valores distintos:

<i>Formato</i>	<i>Descripción</i>	<i>Ejemplo con char c = 'A';</i>
<i>%c</i>	Muestra el carácter	A
<i>%d o %i</i>	Muestra el valor decimal ASCII	65
<i>%o</i>	Muestra el valor en octal	101
<i>%x</i>	Muestra el valor en hexadecimal	41

6. Genera un programa y ejecútalo en la interfaz que elijas con el número binario de tu letra inicial del nombre y realiza un corrimiento a la izquierda y uno a la derecha del bit más significativo.

```
#include <stdio.h>
```

```
int main() {
```

```
    unsigned char letra = 'F'; // Inicial del nombre
```

```
    unsigned char izquierda, derecha;
```

```
printf("Letra: %c\n", letra);  
printf("Valor ASCII: %d\n", letra);  
printf("Binario original: 01000110\n"); // Representación binaria manual
```

```
// Corrimiento del bit más significativo
```

```
izquierda = letra << 1; // Corrimiento a la izquierda
```

```
derecha = letra >> 1; // Corrimiento a la derecha
```

```
printf("\nCorrimiento a la izquierda (<< 1): %d", izquierda);
```

```
printf("\nCorrimiento a la derecha (>> 1): %d\n", derecha);
```

```
printf("\nBinario izquierda: 10001100");
```

```
printf("\nBinario derecha: 00100011\n");
```

```
return 0;
```

```
}
```


Depurador en línea de GDB | Compilador: Código, Compilación, Ejecución, Depuración en línea C, C++

https://www.onlinegdb.com/#

main.c

```
8 *****
9 #include <stdio.h>
10
11 int principal() {
12     char sin signo letra = 'F';  Inicial del nombre
13     sin firmar char izquierda, derecha;
14
15     printf("Letra: %c\n", letra);
16     printf("Valor ASCII: %d\n", letra);
17     printf("Binario original: 01000110\n");  Representación binaria manual
18
19     Corrimiento del bit más significativo
20     izquierda = letra << 1;  Corrimiento a la izquierda
21     derecha = letra >> 1;  Corrimiento a la derecha
22
23     printf("\nCorrimiento a la izquierda (<< 1): %d", izquierda);
24     printf("\nCorrimiento a la derecha (>> 1): %d\n", derecha);
25
26     printf("\nBinario izquierda: 10001100");
27     printf("\nBinario derecha: 00100011\n");
28
29     retorno 0;
30 }
```

Depurador en línea de GDB | Compilador: Código, Compilación, E

https://www.onlinegdb.com/#

main.c

```
8 *****
9 #include <stdio.h>
10
11 int principal() {
12     char sin signo letra = 'F';  Inicial del nombre
13     sin firmar char izquierda, derecha;
14
15     printf("Letra: %c\n", letra);
16     printf("Valor ASCII: %d\n", letra);
17     printf("Binario original: 01000110\n");  Representación binaria manual
18
19     Corrimiento del bit más significativo
20     izquierda = letra << 1;  Corrimiento a la izquierda
21     derecha = letra >> 1;  Corrimiento a la derecha
22
23     printf("\nCorrimiento a la izquierda (<< 1): %d", izquierda);
24     printf("\nCorrimiento a la derecha (>> 1): %d\n", derecha);
25
26     printf("\nBinario izquierda: 10001100");
27     printf("\nBinario derecha: 00100011\n");
28
29     retorno 0;
30 }
```

entrada

Letra: F
Valor ASCII: 70
Binario original: 01000110

Corrimiento a la izquierda (<< 1): 140
Corrimiento a la derecha (>> 1): 35

Binario izquierda: 10001100
Binario derecha: 00100011

...Program finished with exit code 0
Press ENTER to exit console.

Conclusión

Durante esta práctica tuve algunas complicaciones, sobre todo al principio, porque no lograba entender bien algunos conceptos como los tipos de datos y el uso de los corrimientos de bits. Sin embargo, al ir investigando y probando los ejemplos por mi cuenta, poco a poco fui entendiendo mejor cómo funciona el lenguaje C y qué hace cada instrucción. Aun así, reconozco que todavía me falta practicar más para dominar completamente estos temas, pero esta actividad me ayudó a comprenderlos con más claridad y a tener una idea más concreta de cómo se ejecutan los programas.

Referencias

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp. 85-112. Recuperado el 14 de octubre de 2025 de [Laboratorio Salas A y B](#)

Free Software Foundation. (n.d.). Signed and unsigned types (GNU C Language Manual). GNU Project. Recuperado el 14 de octubre de 2025, de https://www.gnu.org/software/c-intro-and-ref/manual/html_node/Signed-and-Unsigned-Types.html

Microsoft. (2023). Diferencias entre procesadores de texto y editores de texto. Recuperado de <https://learn.microsoft.com/>

GNU Project. (2024). Using the GNU Compiler Collection (GCC). Recuperado de <https://gcc.gnu.org/>

Programiz. (n.d.). C program to find ASCII value of a character. Recuperado el 14 de octubre de 2025, de <https://www.programiz.com/c-programming/examples/ASCII-value-character>